Semantic Cache for LLM Serving

Student Name: Pojesh Kumar R
Guide Name: Dr. Kumar R
School Of Computer Science And Engineering

MOTIVATION / INTRODUCTION

- High cost and redundancy in LLM usage: LLMs have significant inference cost ($0.03–$0.06 per 1K tokens) and latency (≈500 ms - 6s), while ~31% of queries are semantically similar indicating strong potential for caching.
- Improved caching approach: ContextualCache replaces global thresholds with per-entry adaptive thresholds for better accuracy and control.

SDG GOAL NUMBER: 9

OBJECTIVES

- Per-entry adaptive thresholds: Conformal similarity thresholds.
- Adaptive learning for new entries: Thompson Sampling bandit with drift detection to tune default thresholds.
- Efficient cache management: Semantic W-TinyLFU admission + CostGDSF eviction to keep valuable entries.
- Optimized retrieval & evaluation: Two-tier lookup (exact + FAISS search), benchmarked against 6 baselines on QA datasets.

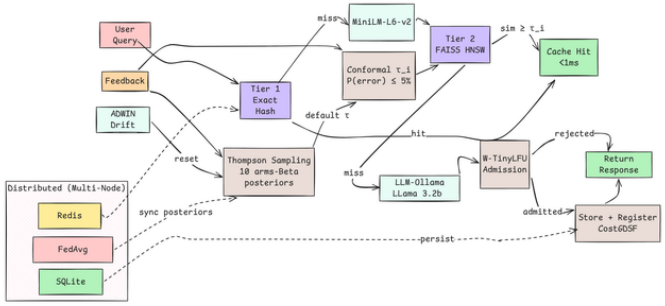
SCOPE OF THE PROJECT

FastAPI-based multi-provider LLM backend with MiniLM embeddings, evaluated on QA datasets, paired with a real-time analytics dashboard and distributed-ready architecture for scalable deployment.

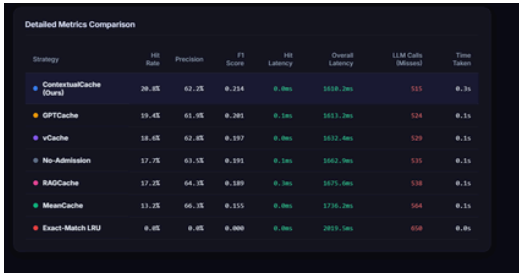
METHODOLOGY

- Tier 1 - Exact Hash Lookup: Query text is normalized and hashed (SHA-256). Checked against in-memory dictionary in <1 ms.
- Tier 2 - Semantic Search: On Tier 1 miss, query is embedded using paraphrase-MiniLM-L6-v2 (384d) and searched against FAISS HNSW index (top-k=10 neighbors).
- Conformal Threshold Check: Each candidate's cosine similarity is compared against its per-entry threshold τ_i, computed as the (1−ε) quantile of nonconformity scores.
- LLM Fallback: On cache miss, query is sent to LLM (Ollama). Response passes through W-TinyLFU admission gate before storage.
- Feedback Loop: Correctness signals update conformal thresholds, Thompson Sampling bandit posteriors, and ADWIN drift detector.

ARCHITECTURE



RESULTS



Detailed Metrics Comparison

Strategy	Hit Rate	Precision	F1 Score	Hit Latency	Overall Latency	LLM Calls (Misses)	Time Taken
ContextualCache (Ours)	20.8%	62.2%	0.214	0.0ms	1610.2ms	515	0.3s
GPTCache	19.4%	61.9%	0.201	0.1ms	1613.2ms	524	0.1s
vCache	18.6%	62.8%	0.197	0.0ms	1632.4ms	529	0.1s
No-Admission	17.7%	63.5%	0.191	0.1ms	1642.9ms	535	0.1s
RAGCache	17.2%	64.3%	0.189	0.3ms	1675.6ms	538	0.1s
MeanCache	13.2%	66.3%	0.155	0.0ms	1736.2ms	564	0.1s
Exact-Match LRU	0.0%	0.0%	0.000	0.0ms	2019.5ms	650	0.0s

CONCLUSION

- Performance gains: ContextualCache outperforms existing methods (GPTCache, MeanCache, vCache, RAGCache) in hit rate and F1, with W-TinyLFU further boosting hit rate by ~3.08%.
- Adaptive accuracy: Per-entry conformal thresholds ensure controlled false positives while Thompson Sampling learns an optimal value for better paraphrase matching.
- Cost efficiency: Enables significant savings, with projected reductions exceeding $8,400/month at 100K queries/day (GPT-4 pricing).

CONTACT DETAILS

MAILID: pojeshkumar.r2022@vitstudent.ac.in

MOBILE NO: 6381200401

REFERENCES

- Regmi & Pun, "GPT Semantic Cache," 2024.
- Gill et al., "MeanCache," 2024.
- Schroeder et al., "vCache," 2025.
- Einziger et al., "TinyLFU," 2017.